SWC3

Software-Entwicklung mit C und C++

Erik Pitzer

WS 2026/27

FH OÖ - Campus Hagenberg - Software Engineering



10. C++ Grundlagen

Primitive Datentypen und Literale

- sehr ähnlich wie C, aber etwas *strenger*
- `bool` mit `true` und `false`
- [`signed`|`unsigned`] `char`
  - Literale (z.B. `'c'`) sind vom Typ `char` (nicht `int` wie in C)
  - `char8_t`, `char16_t`, `char32_t` für UTF-8, -16 und -32
- ganze Zahlen (*integer*) (`short`|`long`|`long long`) (`signed`|`unsigned`) `int`
  - wieder Implementationsabhängig aber mit Mindestgrößen
    - `sizeof(char) >= 1`, `sizeof(int) >= 2`, `sizeof(long) >= 4`, `sizeof(long long) >= 8`
- Fließkommazahlen `float`, `double`, `long double` gleich wie in C
  - seit C++23 auch exakte Bitbreite in `<stdfloat>`
    - `float16_t`, `float32_t`, `float64_t`, `float128_t`, `bfloat16_t`
- Größenrelationen werden durch typische Datenmodelle beschrieben, z.B.
  - `ILP32`: `int`, `long` und Zeiger haben 32 Bits
  - `LLP64`: `int` und `long` 32 Bit, Zeiger haben 64 Bits

Größe und Wertebereiche Abfragen

- Abfrage von Wertebereich und Größe
  - wie in C durch Makros in `<limits.h>` bzw. `<climits>`
  - C++ Schnittstelle über Schablonen in `<limits>`

```cpp
#include <print>
#include <limits>

int main() {
  using int_limits = std::numeric_limits<int>;
  std::println("properties of int");
  std::println("- range: {} .. {}", int_limits::lowest(), int_limits::max());
  std::println("- signed: {}", int_limits::is_signed);
  std::println("- has infinity: {}", int_limits::has_infinity);
  std::println("- significant decimal digits: {}", int_limits::digits10);
}
```

```
properties of int
- range: -2147483648 .. 2147483647
- signed: true
- has infinity: false
- significant decimal digits: 9
```

Abgeleitete Datentypen

- Wie in C
  - Zeiger (*pointer*), `int* p`
  - Feld (*array*), `int a[]`
  - Vereinigungstyp, `union number { int16_t i; float16_t f; }`
  - Typalias, `typedef int* int_ptr_t`
- Ähnlich zu C
  - Aufzählungen `enum Color : uint8_t { red, green, blue }`
    - ohne eigenen Tag-Namespace, Typname ist einfach nur `Color`
    - innere Bezeichner (`red`, `green`, `blue`) sind global verfügbar
  - Scoped Enums `enum class Color : uint8_t { red, green, blue }`
    - Bezeichner, z.B. `Color::red` sind Elemente des Typs
  - Funktionsnamen, z.B. `void increment(int &x);` erlauben Überladung
    - mehrere Funktionen mit gleichem Namen (aber unterschiedlicher Schnittstelle) möglich
- neu in C++
  - Typalias, z.B. `using IntPtr = int*`
  - Referenztypen, `int &x = a`
    - ähnlich zu Pointer aber nicht “verbiegbar” und garantiert nicht `nullptr`
  - Klassen, `class Color { /* ... */ }`
    - nicht nur Datenkapsel (wie C `struct`)
    - enthält auch Funktionalität in Form von “Methoden” wie in Java

Typumwandlung (*type cast*) (i)

- automatische (implizite) Typumwandlung, z.B. bei Parameterübergabe oder Zuweisung einer Variable

```
long l = 12345678901234567891;
int i = l; // warning: conversion from 'long int' to 'int' may change value [-Wconversion]
SHOW(l, i);
```

```
l = 1234567890123456789; i = 2112454933
```

- Änderung der “Konstantheit” (*constness*)

```
const double G = 2.2e-5;
SHOW(G);
const double &G_ref = G;
const_cast<double&>(G_ref) = 1.1e-10; // NOTE: no error but no effect
SHOW(G);
```

```
G = 2.2e-05
G = 2.2e-05
```

Typumwandlung (*type cast*) (ii)

- explizite Typumwandlung zum *“bestmöglich”* entsprechenden Wert

```
double d = -1.1;
unsigned u = static_cast<unsigned>(d);
bool b = static_cast<bool>(d);
SHOW(d, u, b);
```

```
d = -1.1; u = 4294967295; b = true
```

- neue (kompatible) “Interpretation” der Bits, z.B. Array als Adresse

```
double a[] = {1, 2, 3};
long addr = reinterpret_cast<long>(a);
SHOW(a);
std::println("addr = 0x{:x}", addr);
// long l = reinterpret_cast<long>(a[0]); // XXX: compiler error
```

```
a = 0x7bb317af0080
addr = 0x7bb317af0080
```

Typumwandlung (*type cast*) (iii)

- C-style cast `(int)1.1`
  - wenn möglich durch anderen Cast-Typ ersetzen
- in späteren Kapiteln kommen noch
  - `dynamic_cast<>()` für Umwandlung innerhalb einer Klassenhierarchie
  - benutzerdefinierte Umwandlung durch Überladen eines eigenen Operators

Variablen

- nur `default`-initialisiert wenn global

```
int g;

void f() {
  int i; // NOTE: uninitialized
  SHOW(i);
}

int main() {
  SHOW(g);
  int j; // NOTE: uninitialized
  SHOW(j);
  f();
}
```

```
g = 0
j = -16843010
i = -16843010
```

- besser explizite Default-Initialization mit geschwungenen Klammern »{}«

```
int g {};

void f() {
  int i {};
  SHOW(i);
}

int main() {
  SHOW(g);
  int j {};
  SHOW(j);
  f();
}
```

```
g = 0
j = 0
i = 0
```

Default-initialization - cppreference.com

Typinferenz

- Compiler kann oft Typ automatisch bestimmen, Schlüsselwort `auto`
  - nur verwenden wenn Typ ohnehin klar ist

```
auto i = (int)1;

int f(int i) {
  auto j = i;
}
```

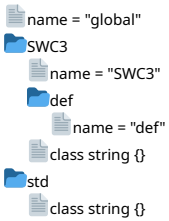
Namensräume (*namespaces*)

- weitere Unterteilung von Symbolen in Namensräume (ähnlich zu Paketen in Java)
  - einfache Unterscheidbarkeit (verringert Aliasing)
  - Trennung durch *Scope-Resolution-Operator* »::«
  - keine Übereinstimmung mit Datei- oder Verzeichnisname notwendig (anders als Java)

```
std::string name = "global";
namespace SWC3 {
  std::string name = "SWC3";
  namespace def {
    std::string name = "def";
  }
  class string {}; // no conflict with std::string
}

int main() {
  SHOW(name);
  SHOW(SWC3::name);
  SHOW(SWC3::def::name);
  using SWC3::name;
  SHOW(name);
  SHOW(::name);
}
```

```
name = "global"
SWC3::name = "SWC3"
SWC3::def::name = "def"
name = "SWC3"
::name = "global"
```



switch [[fallthrough]]

- Hinweis an Compiler, dass `[[fallthrough]]` Absicht ist

```
std::string to_string(int x) {
  switch (x) {
    case 0: return "none";
    case 1: return "one";
    case 2: [[fallthrough]];
    case 3: [[fallthrough]];
    case 4: return "several";
    default: return "many";
  }
}
int main() {
  SHOW(to_string(1));
  SHOW(to_string(3));
  SHOW(to_string(10));
}
```

```
to_string(1) = "one"
to_string(3) = "several"
to_string(10) = "many"
```

Default-Parameter

- C++ erlaubt die Angabe von Defaultwerten für Parameter
- können beim Aufruf (von hinten beginnend) weggelassen werden
- Defaults am besten bei der Deklaration angeben

```
void print_all(int a, int b = 10, int c = 20);

int main() {
  print_all(0, 0, 0);
  print_all(0, 0); // c = default
  print_all(0); // b, c = default
  // print_all(); // XXX: error no default for a
}

void print_all(int a, int b, int c) {
  SHOW(a, b, c);
}
```

```
a = 0; b = 0; c = 0
a = 0; b = 0; c = 20
a = 0; b = 10; c = 20
```

Funktionen überladen

- C++ erlaubt wie C globale Funktionen (ohne Klassen)
- C++ erlaubt auch Funktionen zu überladen
  - Funktion mit gleichem Namen
  - aber unterschiedlicher Anzahl oder Typ von Parametern
  - Unterscheidung durch Rückgabewert ist nicht möglich

```
// file: name_mangling.cpp
#include <print>

void ovld(int i) { PL("int {}", i); }
void ovld(double & d) { PL("dbl {}", d); }
void ovld(double const& d) {
  PL("const double {}", d);
}

int main() {
  ovld(1);
  ovld(1.1);
}
```

```
int 1
const double 1.1
```

- Funktionen werden intern umbenannt (*“name mangling”*, compilerabhängig)
- Parametertypen werden Teil des Namens, z.B.

```
g++ -std=c++23 -c name_mangling.cpp
nm name_mangling.o | grep ovld
```

```
0000000000000000 T _Z4ovldi
000000000000005b T _Z4ovldRd
00000000000000b7 T _Z4ovldRKd
```

extern C

- um C++ Funktionen in C zu verwenden muss *name mangling* mit `extern "C"` deaktiviert werden

```
// file: extern_c.c

void my_cxx_function(int i, double d) {}

#ifdef __cplusplus
extern "C"
#endif
void my_c_function(int i, double d) {}
```

- C++ Compiler

```
g++ -std=c++23 -c extern_c.c
nm extern_c.o | grep function
```

```
000000000000000f T my_c_function
0000000000000000 T _Z15my_cxx_functionid
```

- C compiler

```
gcc -std=c23 -c extern_c.c
nm extern_c.o | grep function
```

```
000000000000000f T my_c_function
0000000000000000 T my_cxx_function
```

Referenzen und Referenzparameter

- Variablen & Parameter können in C++ auch Referenztyp sein
  - Datentyp ist z.B. `int&` (ähnlich zu Zeiger `int*`)
  - garantiert nicht `nullptr`
  - Referenzsemantik bei Aufruf nicht sichtbar (kein Unterschied zu call-by-value)

```
void inc1(int &i) {
  i++;
}

void inc2(int i) {
  i++;
}

int main() {
  int i {};
  SHOW(i);
  inc1(i); // NOTE: pass by reference
  SHOW(i);
  inc2(i); // NOTE: pass by value
  SHOW(i); // i not incremented
}
```

```
i = 0
i = 1
i = 1
```

Ein- und Ausgabe vor C++23

- vor C++23 Ein- und Ausgabe über Streams und Operatoren (`operator <<`)
- Standard-Input (`std::cin`) und -output (`std::cout`) im Header <iostream>
- Datei I/O im Header <fstream> (file stream)

```cpp
// output operators <<
std::cout << 1 << " string " << 2.3 << std::endl;

std::ifstream f { "/dev/urandom" };
size_t counter = 10;
char c;
// input operators >>
while (f >> c && counter-- > 0) {
  std::cout << static_cast<int>(c) << ", ";
}
```

```
1 string 2.3
-107, 94, -29, -123, 28, 42, -68, 67, -109, -36,
```

- Vorteile
  - hohe Typsicherheit
    - jeder Datentyp hat eigenen Ausgabeoperator
  - hohe Flexibilität
    - einfaches Hinzufügen von eigenen Typen
    - Streams für Console, Datei, und weitere Situationen, z.B. Container
- Nachteile
  - sehr ungewöhnlich
  - schlechtere Lesbarkeit

Ein- und Ausgabe seit C++23 (i)

- eigene `std::print()` und `std::println()` Funktionen mit Formatangabe[1]
- aufbauend auf `std::formatter` und `std::format()` von C++20
- Formatieren von Containern bereits in Standardbibliothek
- optionale Nummer des Arguments

```
int a[] = {1, 2 ,3};
std::println(" {},  {},  {},  {}", 1, "string", 2.2, a);
std::println("{3}, {2}, {1}, {0}", 1, "string", 2.2, a);
```

```
 1,  string,  2.2,  [1, 2, 3]
[1, 2, 3], 2.2, string, 1
```

- Angabe von Breite (`"{:10}"`) und Genauigkeit (`"{:.5}"`) (Anzahl signifikanter Stellen)

```
std::println("|{:10.5}|", 123.456);
```

```
|    123.46|
```

[1] Standard format specification (since C++20) - cppreference.com

Ein- und Ausgabe seit C++23 (ii)

- Angabe von Textausrichtung (linksbündig `"{:<}"`, zentriert `"{:^}"` oder rechtsbündig `"{:>}"`)

```
std::println("| {0:<5} | {0:^5} | {0:>5} |", 1);
```

```
| 1     |   1   |     1 |
```

- Vorzeichen erzwingen `"{:+}"` oder mit Nullen auffüllen `"{:0}"`

```
std::println("{0:+} | {0:05}", 1);
```

```
+1 | 00001
```

- hexadezimale (`"{:x}"`), oktale (`"{:o}"`) und binäre (`"{:b}"`) Ausgabe
  - mit (z.B. `"{:#x}"`) oder ohne Präfix

```
std::println("{0:#x}, {0:#o}, {0:#b}", 10);
std::println("{0:x}, {0:o}, {0:b}", 10);
```

```
0xa, 012, 0b1010
a, 12, 1010
```

C++ Felder und Vektoren `std::array` & `std::vector`

```cpp
#include <vector>
#include <array>

void print_vector(std::vector<int> v) {
  SHOW(v, v.size());
}

int main() {
  std::array<int, 3> a {};
  SHOW(a, a.size());
  a[2] = 2;
  SHOW(a);

  std::vector<int> v {1, 2, 3};
  print_vector(v);

  std::vector<int> v2 {v};
  v2.push_back(4);
  SHOW(v, v2);
  v = v2;
  SHOW(v, v2);
}
```

- moderne Alternativen zu C-style array
  - `std::array` kennt eigene Größe
  - `std::vector` kann sogar Größe ändern
- Elementtyp und Array-Größe wird als Template-Parameter in spitzen Klammern angegeben
- Arrays unterschiedlicher Länge haben unterschiedlichen Typ
- Vektoren haben gleichen Typ, solange Element-Typ gleich
- erlauben auch direkte Zuweisung mit Index-Operator

```
a = [0, 0, 0]; a.size() = 3
a = [0, 0, 2]
v = [1, 2, 3]; v.size() = 3
v = [1, 2, 3]; v2 = [1, 2, 3, 4]
v = [1, 2, 3, 4]; v2 = [1, 2, 3, 4]
```

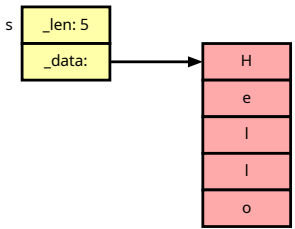
Standard library header <array> (C++11) - cppreference.com
Standard library header <vector> - cppreference.com

C++ Zeichenkettenpuffer `std::string`

- moderne Alternative zu C-Zeichenketten `std::string`
  - verwaltet eigene Größe
  - kann verändert werden
  - Verwendung mit Wertesemantik (*value type*)
  - automatische Speicherverwaltung im Hintergrund
  - viele Operationen und eigene Operatoren
  - nicht Teil der Sprache, aber Teil der Standardbibliothek (Header <string>)

```cpp
std::string s {"Hello"}; // Aufruf mit `const char*`
SHOW(s);
SHOW(s.length(), s.size());
std::string s2 = s + " World"; // concat with `+`
s2 += "!"; // append with `+=`
SHOW(s, s2);
// std::string s3 = s2 + 3; // ERROR: string + int
```

```
s = "Hello"
s.length() = 5; s.size() = 5
s = "Hello"; s2 = "Hello World!"
```

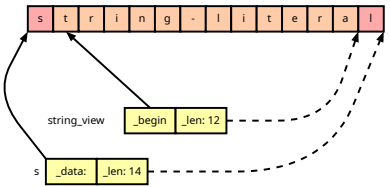


C++ Zeichenketten-Sichten `std::string_view`

- seit C++17 Klasse für Zeichenketten-“Sichtfenster” `std::string_view`
  - erlauben selbst nur lesenden Zugriff auf Zeichenkette
  - dahinter liegende Zeichenkette kann sich ändern
  - sehr leichtgewichtig (nur Zeiger + Länge)
  - kann weiter eingeschränkt werden

```
std::string_view l { "literal"};
SHOW(l);
std::string s { "string " };
s += l;
SHOW(s);
std::string_view v { s };
SHOW(v);
s[6] = '-'; // change of underlying string
SHOW(v);
// v[3] = 'i'; // XXX: error read-only view
v.remove_prefix(1);
v.remove_suffix(1);
SHOW(v);
```

```
l = "literal"
s = "string literal"
v = "string literal"
v = "string-literal"
v = "tring-litera"
```



Bereichsschleifen (*ranged-based loops*)

- Container oder Zeichenketten mit speziellem Syntax durchlaufen
  - keine explizite Größenangabe notwendig

```
std::string s = "SWC";
for (char c : s) {
  SHOW(c);
}
```

```
c = 'S'
c = 'W'
c = 'C'
```

```
std::vector<int> v = {1, 2, 3};
for (const auto &e : v) {
  SHOW(e);
}
```

```
e = 1
e = 2
e = 3
```

dynamischer Speicher (i)

```
struct X {
  char x {'x'};
};

X *x = new X();
SHOW(x->x);
delete x;

X *xs = new X[10]();
SHOW(xs[1].x);
// delete xs; // XXX: wrong delete operator
delete[] xs;

X *xs2 = new X[10]();
SHOW(xs2[3].x);
delete[] xs2;
```

- neue Operatoren für dynamische Speicherverwaltung
  - `new` zum Reservieren von Speicher und initialisieren von Ressourcen eines Typs
  - `delete` zum Freigeben von Ressourcen und Speicher eines Typs
  - `new[]` zum Reservieren eines Feldes
  - `delete[]` zum Freigeben eines Feldes
  - nicht mit `malloc()` und `free()` vermischen

```
x->x = 'x'
xs[1].x = 'x'
xs2[3].x = 'x'
```

dynamischer Speicher (ii)

- uninitialisiertes Array durch Weglassen von Klammern

```
int *a = new int[10]();
int *b = new int[10]{};
int *c = new int[10]; // NOTE: not initialized
SHOW(a[5], b[5], c[5]);
delete[] a;
delete[] b;
delete[] c;
```

```
a[5] = 0; b[5] = 0; c[5] = -1094795586
```

Wertesemantik

- in modernem C++, zunehmend Verzicht auf Zeiger und Referenzen
- Übergabe immer per Wert (*by value*)
  - verhindert viele Fehler
  - auf ersten Blick ineffizient (häufiges Kopieren)
  - durch Move-Semantik oft aber effizient nutzbar (siehe später)

```
std::string concat(std::string a, std::string b) { /* ... */ }
```

- alternativ Übergabe per konstanter Referenz (*const reference*)
  - Referenz ist nicht-leerer Zeiger
    - es wird nur die Adresse übergeben
  - erlaubt (meistens !) nur lesenden Zugriff

```
std::string concat(const std::string &a, const std::string &b) { /* ... */ }
```